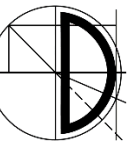


Grundlagen der Programmierung

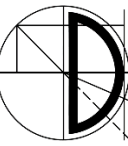
Kapitel 11: Klassen (Teil 1)

Prof. Dr. Samuel Kounev, Martin Sträßer M.Sc., Daniel Grillmeyer M.Sc.

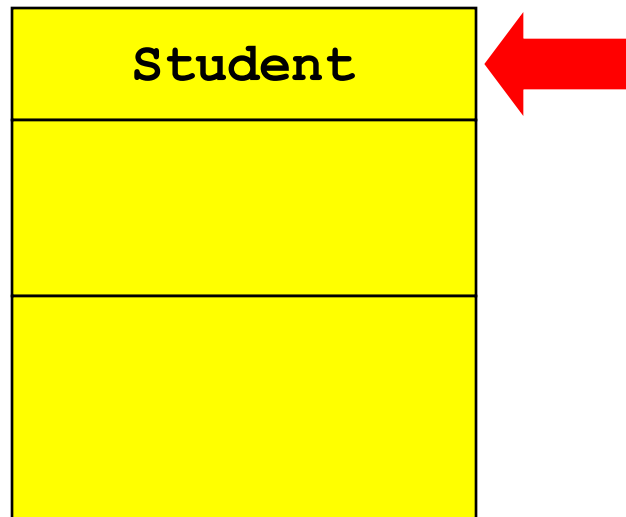
- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich

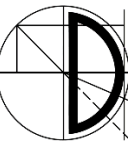


- Instanzvariablen
- Konstruktoren
- Instanzmethoden

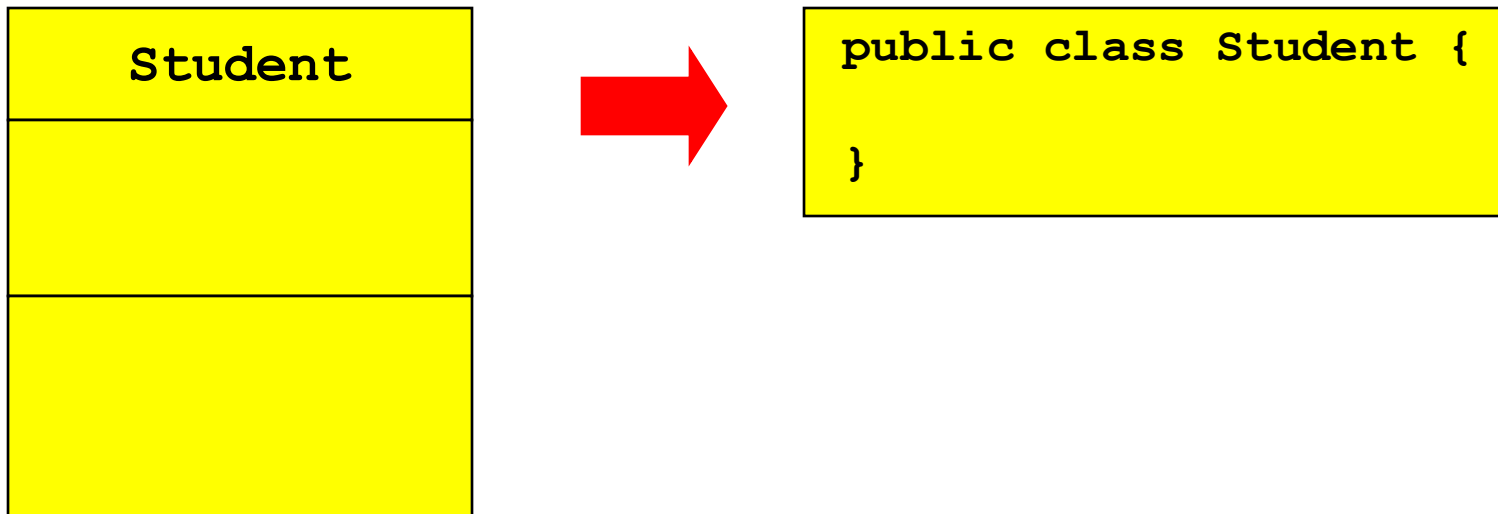


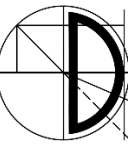
- Wir werden nun unsere erste Klasse von Objekten modellieren: die Klasse **Student**
- Zu diesem Zweck werden wir auf der einen Seite stets ein graphisches Modell der Klasse in UML betrachten...





- ... und auf der anderen Seite einen Blick auf die entsprechende Realisierung in Java werfen

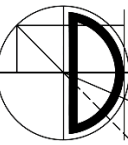




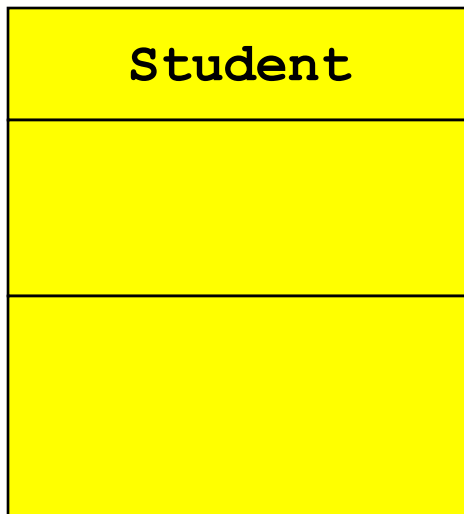
- Unser zweizeiliger Java-Code stellt bereits eine gültige Klassenbeschreibung dar
- Wir können die Klasse bereits übersetzen und **instanziierten** - also Objekte aus ihr erzeugen

Student

```
public class TestProgramm {  
    // Meine main-Methode  
    public static void main(String[] args) {  
        // Erzeuge ein Objekt  
        Student charly = new Student();  
    }  
}
```

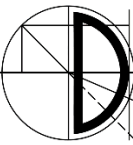


- Die Anweisung innerhalb der `main`-Methode ist für uns nicht neu
- Sie dient der **Instanziierung**, erzeugt also ein **Objekt**

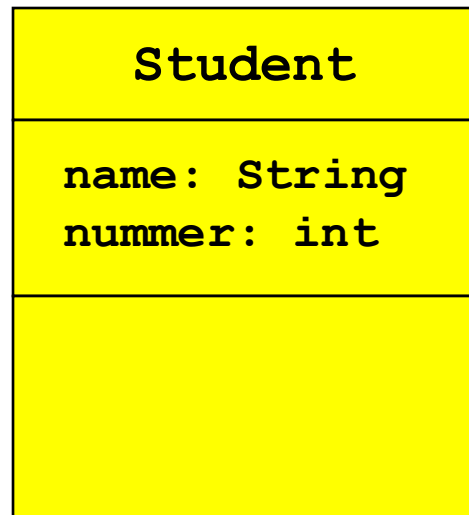


```
public class Student {  
  
}
```

```
Student charly = new Student();
```



- Unsere Klasse soll in der Lage sein, Name und Matrikelnummer eines Studenten im zugehörigen Objekt zu hinterlegen
- Wir tragen unsere Erweiterungen in das UML-Klassendiagramm ein - hierbei stehen Daten immer im oberen der beiden Rechtecke

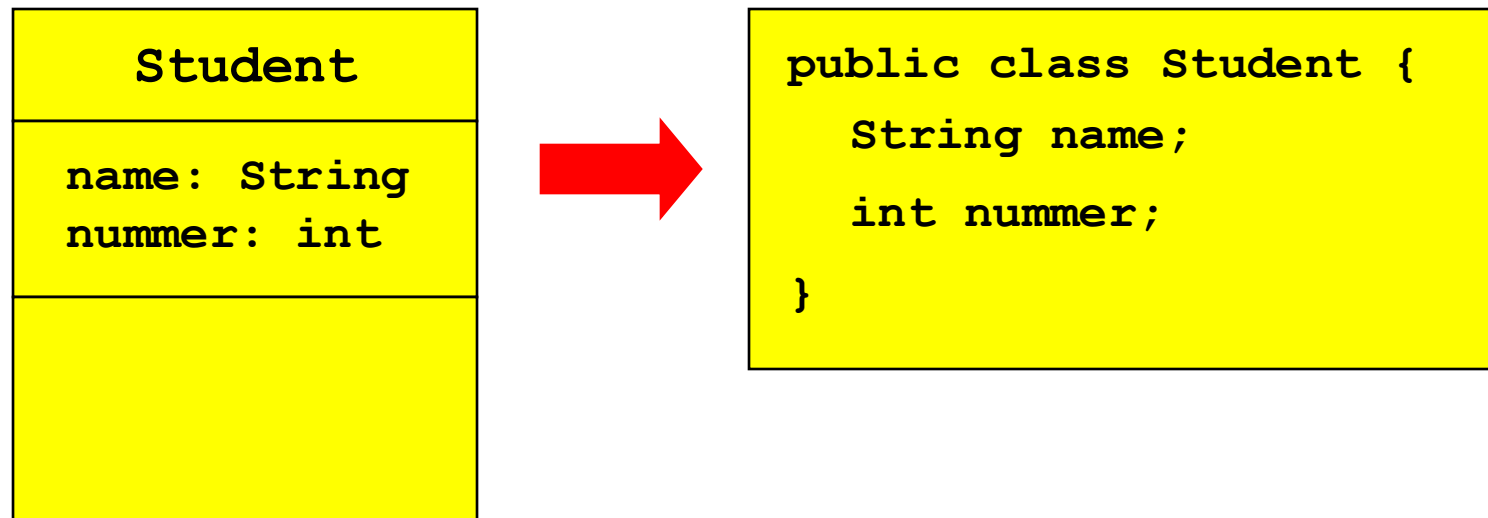


```
public class Student {  
  
}  

```

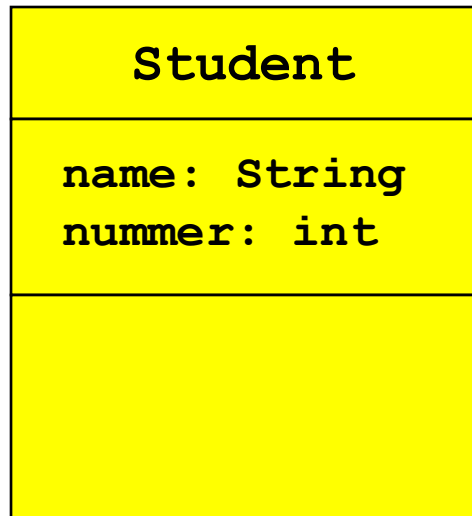


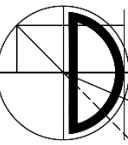

- Die so definierten Variablen werden beim Erzeugen eines neuen **Student**-Objekts für dieses Objekt automatisch miterzeugt und können vom Benutzer individuell belegt werden
- Sie gehören also der jeweiligen Instanz und werden deshalb als **Instanzvariablen** bezeichnet





- Beispiel: Wir wollen in unserem „Testprogramm“ einen Studenten namens „Kalle Karlsson“ mit der Matrikelnummer **978432** erstellen. Der Variablenname des Objekts soll hierbei **charly** lauten.



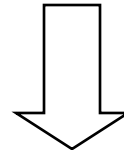


Der Zugriff auf Instanzvariablen erfolgt in Java in der Form
<Objektname>.<Variablenname>

Student

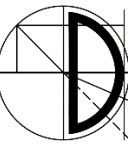
name: String
nummer: int

```
Student charly = new Student();
```

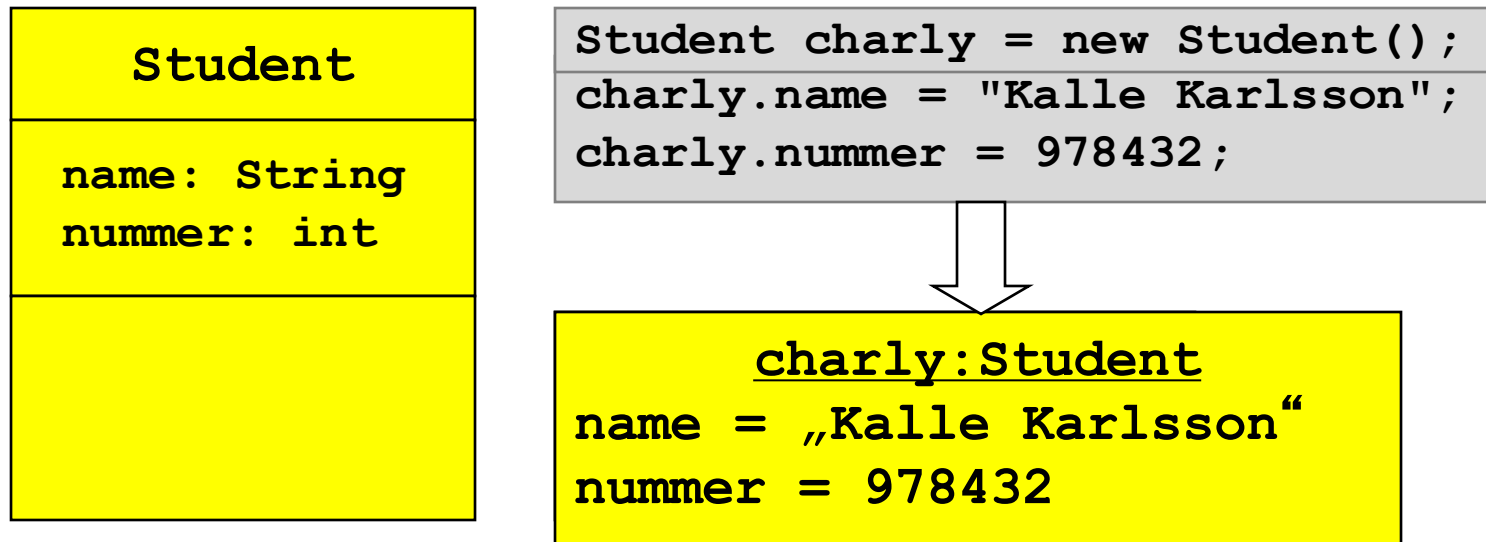


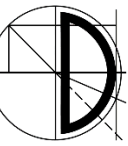
charly:Student

name = ?
nummer = ?

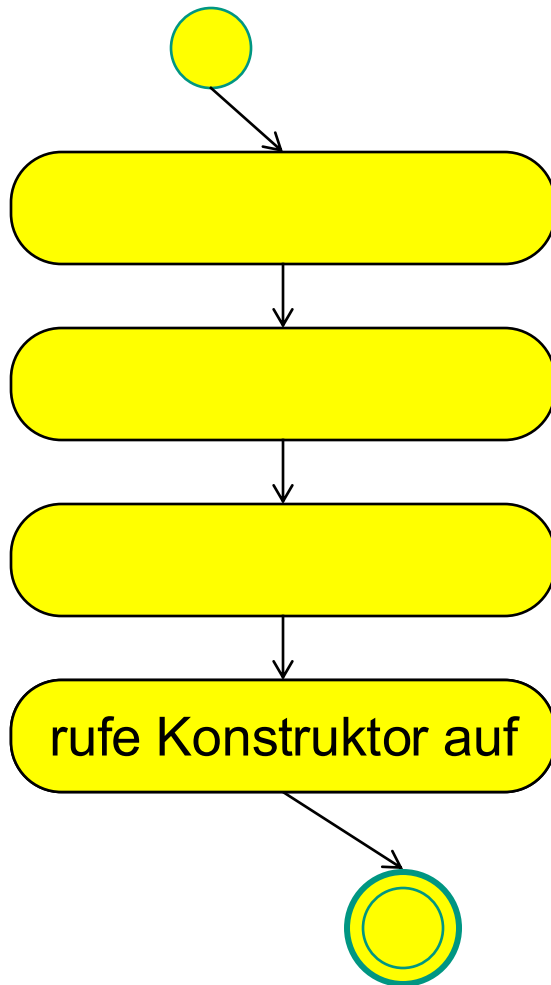
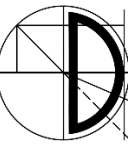


- Wir können die Werte in **name** und **nummer** des Objekts **charly** durch einfache Wertzuweisungen manipulieren

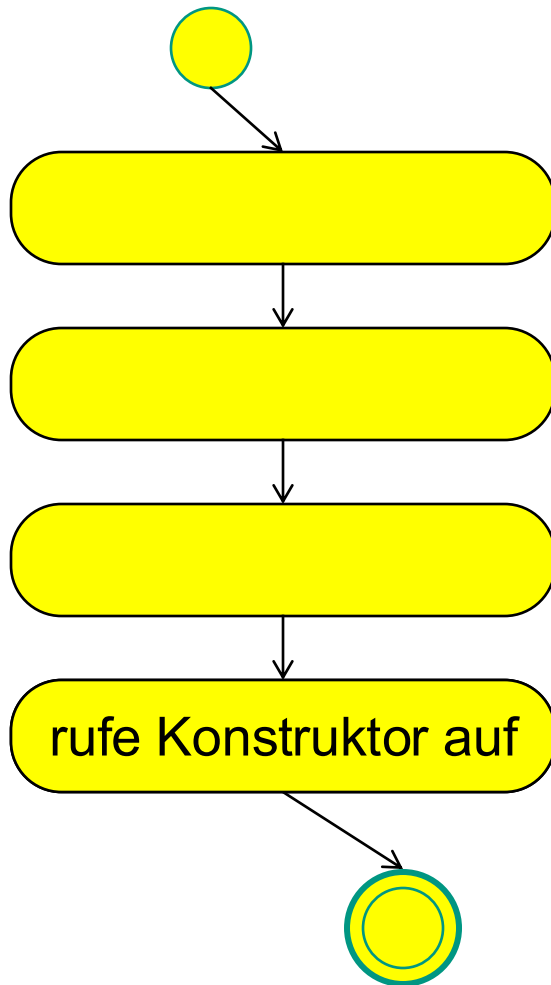
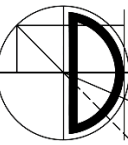




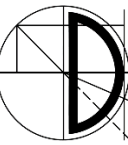
- Instanzvariablen
- **Konstrukturen**
- Instanzmethoden



- Auch in Java fallen Objekte nicht einfach „vom Himmel“, sondern werden nach einem fest vorgegebenen Schema erstellt
- Wir wollen an dieser Stelle nicht auf jede einzelne dieser Aktivitäten eingehen; für uns ist lediglich der letzte Schritt von Interesse: der **Aufruf des Konstruktors**



- Konstrukturen kommen erst am Ende eines Instantiierungsprozesses ins Spiel. Das Objekt ist prinzipiell schon erzeugt: Speicher wurde belegt, Instanzvariablen vereinbart, gewisse Initialisierungen bereits vorgenommen...
- Es ist nun Aufgabe des Konstruktors, Initialisierungsarbeiten zu erledigen, die das System nicht „automatisch“ durchführen kann
- *Hierzu zählt unter anderem das Belegen von Instanzvariablen mit vom Benutzer vorgegebenen Startwerten*



- Werfen wir erneut einen Blick auf unsere Klasse **Student**
- In unserem momentanen Entwicklungsstadium haben wir in der Klasse noch keinen Konstruktor definiert. *Das System benötigt jedoch unbedingt einen Konstruktor, um Objekte bilden zu können!*
- Damit also Java dennoch in der Lage ist, die Klasse zu instantiieren, wird bei der Übersetzung vom Compiler in der Klassendatei automatisch ein „leerer Konstruktor“ angefügt, dessen einzige Aufgabe es ist, „nichts“ zu tun. Wir bezeichnen diesen Konstruktor in Zukunft als den **Standard-Konstruktor** bzw. den **Default-Konstruktor**

```
public class Student {  
    String name;  
    int nummer;  
  
}
```



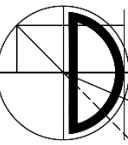

- Würden wir den „leeren“ Default-Konstruktor in unseren Quellcode aufnehmen, hätte dieser die folgende Gestalt:
- Es fällt eine gewisse Ähnlichkeit mit der Vereinbarung von Methoden auf. Wir stellen jedoch fest, dass
 - der Name des Konstruktors kein beliebiger Bezeichner sondern der Name der Klasse ist
 - kein Rückgabetypp angegeben wurde (nicht einmal void)

```
public class Student {  
    String name;  
    int nummer;  
  
    Student() {  
    }  
  
}
```



- Kommen wir zurück zu unserer Klasse **Student**
- Anstelle des Default-Konstruktors wollen wir uns einen eigenen Konstruktor definieren
- Diesem wollen wir Namen und Matrikelnr. eines Studenten als Argumente übergeben können
- In diesem Fall ist also der Name der Klasse wieder **Student**
- Die Liste der Argumente ist **String n**, **int nr**

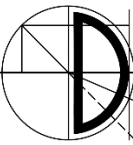
```
public class Student {  
    String name;  
    int nummer;  
  
    Student(String n, int nr) {  
    }  
  
}
```



- Erinnern wir uns nun daran, wie wir unsere Objekte bisher mit Werten initialisiert haben
- Um die einzelnen Werte setzen zu können, haben wir uns eines Zugriffs der Form `<Objektname>.<Variablenname>` bedient
- **Wie** sollen wir aber innerhalb des Konstruktors wissen, ob unser Objekt nun **charly**, **lena** oder **lassy** heißt?

```
public class Student {  
    String name;  
    int nummer;  
  
    Student(String n, int nr) {  
  
    }  
}
```

```
Student charly = new Student();  
charly.name = "Kalle Karlsson";  
charly.nummer = 978432;
```

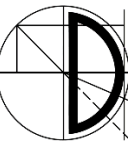


- Innerhalb eines Objektes können wir das Objekt selbst mit dem Schlüsselwort **this** ansprechen

Der Zugriff auf Instanzvariablen innerhalb des Konstruktors erfolgt somit in der Form
this.<Variablenname>

```
public class Student {  
    String name;  
    int nummer;  
  
    Student(String n, int nr) {  
  
    }  
}
```

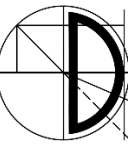
```
Student charly = new Student();  
charly.name = "Kalle Karlsson";  
charly.nummer = 978432;
```



- Wollten wir also innerhalb des Konstruktors die Instanzvariablen **name** und **nummer** auf „Kalle Karlsson“ und 978432 setzen, so könnten wir dies in der rechts angegebenen Form bewerkstelligen.
- Genau genommen können wir das **this** auch weglassen. Da es im Konstruktor keine lokale Variable **name** oder **nummer** gibt, "weiß" die Methode trotzdem, dass sie die Instanzvariablen des eigenen Objekts ansprechen muss!

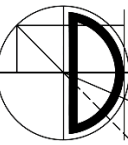
```
public class Student {  
    String name;  
    int nummer;  
  
    Student(String n, int nr) {  
        this.name = "Kalle Karlsson";  
        this.nummer = 978432;  
    }  
}
```

```
Student charly = new Student();  
charly.name = "Kalle Karlsson";  
charly.nummer = 978432;
```



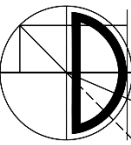
- Natürlich wollen wir unseren Konstruktor allgemeiner halten und ihm die Werte aus der Argumentliste (die Werte von **n** und **nr**) zuweisen
- Das Vorgehen ist hierbei jedoch vollkommen identisch

```
public class Student {  
  
    String name;  
    int nummer;  
  
    Student(String n, int nr) {  
        this.name= n;  
        this.nummer= nr;  
    }  
  
}
```

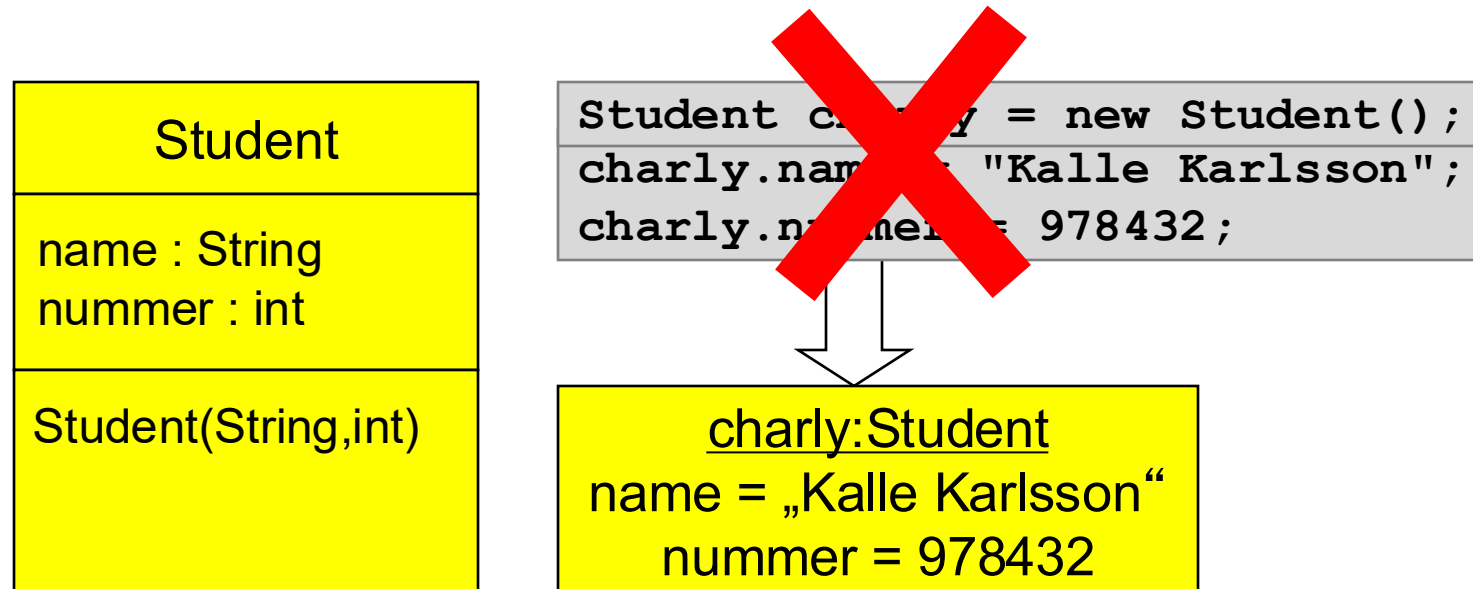


- Anstelle der Parameter **n** und **nr** könnte der Konstruktor auch die Parameter **name** und **nummer** haben, und alles würde funktionieren. Dann könnten wir jedoch das **this** nicht weglassen, da es benötigt wird, um die lokalen Variablen bzw. die Parameter von den Instanzvariablen zu unterscheiden!
- Dies ist auch best practice, da die Namen der Parameter und der Instanzvariablen gleich sind

```
public class Student {  
  
    String name;  
    int nummer;  
  
    Student(String name,  
            int nummer) {  
        this.name = name;  
        this.nummer = nummer;  
    }  
  
}
```



- Hierbei tragen wir den Konstruktor in das untere Feld unseres Klassendiagrammes ein
- Wir haben in unserer Klasse einen eigenen Konstruktor definiert. Beim Übersetzen unseres Quelltextes wird der Compiler also keinen Standardkonstruktor mehr einfügen. Der unten angegebene Aufruf des **new**-Operators ist somit also **falsch!**





- Um unser Objekt **charly** also zu erzeugen, müssen wir dem Operator die von uns gewünschten Initialwerte als Argumente übergeben. Diese werden dann an den Konstruktor weitergeleitet.

Student
name : String nummer : int
Student(String,int)

```
Student charly =  
    new Student("Kalle Karlsson", 978432) ;
```

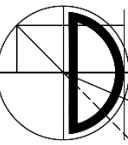
```
charly:Student  
name = „Kalle Karlsson“  
nummer = 978432
```



- Wir können also mit Hilfe des Konstruktors nicht nur Objekte erzeugen, sondern diesen auch gleich die gewünschten Startwerte zuweisen. Statt drei Befehlen benötigen wir nun nur noch einen!
- Diese Ersparnis kommt uns beim Programmieren insbesondere dann zugute, wenn wir viele Instanzen unserer Klasse erzeugen wollen.

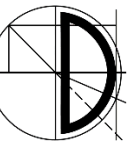
Student
name : String nummer : int
Student(String,int)

```
Student charly =  
    new Student("Kalle Karlsson",978432);  
Student lena =  
    new Student("Lena Lustig",888808);  
Student rudi =  
    new Student("Rudi Rentier",117704);  
Student marion =  
    new Student("Marion Maus",123456);  
Student jessica =  
    new Student("Jessica Java",9875632);
```

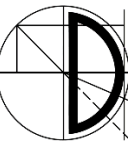


- Ähnlich wie bei Methoden ist es natürlich auch möglich, **Konstrukturen zu überladen**. Wir können mehrere Konstrukturen mit unterschiedlicher Argumentliste definieren, so dass der Benutzer je nach Bedarf zw. verschiedenen Alternativen wählen kann.
- Eine mögliche Anwendung hierfür wäre beispielsweise, den inzwischen verschwundenen Default-Konstruktor zu ersetzen. Wir könnten einen neuen Konstruktor mit leerer Argumentliste definieren, der entweder nichts tut oder die Instanzvariablen auf irgendeinen Standardwert setzt.

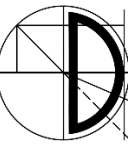
Student
name : String nummer : int
Student(String,int) Student()



- Instanzvariablen
- Konstruktoren
- **Instanzmethoden**

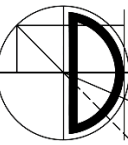


- Eine Instanzmethode ist eine Methode, die **zu einer speziellen Instanz** einer Klasse (also einem Objekt) gehört
- Instanzmethoden werden (wie auch Instanzvariablen) innerhalb der Klassendefinition vereinbart
- Alle in den vorherigen Lektionen erlernten „Handgriffe“ im Umgang mit Methoden (Überladen, rekursive Definition,...) lassen sich auch auf Instanzmethoden problemlos anwenden

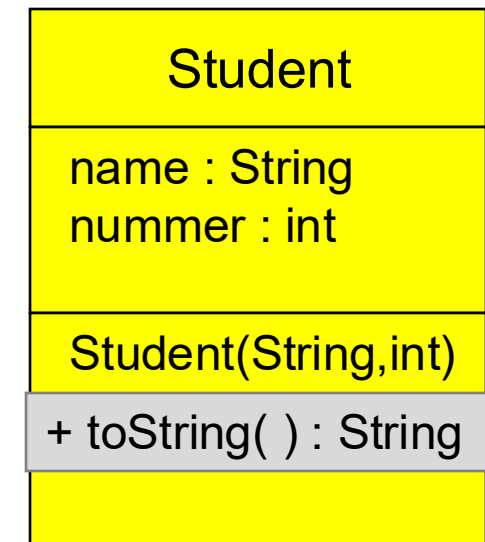


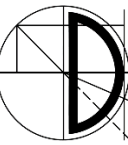
- Beispiel: Wir wollen unsere Klasse **Student** so erweitern, dass die in ihr gespeicherten Daten in aufbereiteter Form dargestellt werden können.
- Hierbei soll diese „Aufbereitung“ einen String als Ergebnis haben, der in der Form **student <name> hat die Matrikelnummer <nummer>** gestaltet sein soll.

Student
name : String nummer : int
Student(String,int)



- Wir wollen eine derartige Aufbereitung „auf Befehl“ ausführen können und entwerfen sie deshalb als Methode
- In unserem UML-Modell werden Methoden in das untere Feld eingetragen
- Wir geben der Methode den Namen `toString`, da wir unser Studenten-Objekt in eine Darstellung als String umwandeln wollen
- Wir wollen einen String als Ergebnis zurückgeben





- Wir wollen die Methode als Instanzmethode entwerfen, das heißt sie gehört zu dem zu bearbeitenden Objekt und hat somit bereits vollen Zugriff auf die Daten. Wir brauchen ihr also keine Argumente zu übergeben.

Student
name : String nummer : int
Student(String,int) + toString() : String

```
public class Student {  
    ...  
  
    public String toString() {  
        return "Student "  
            + this.name  
            + " hat die Matrikelnummer "  
            + this.nummer  
            + ".";  
    }  
} // Ende der Klasse
```




- Der Kopf der Methode entspricht im Prinzip den von uns bisher definierten Methoden - mit der Ausnahme, dass wir das **Schlüsselwort `static` weggelassen** haben. Wir werden jedoch erst später erfahren, welche Bedeutung dieses Wort genau hat.

Student
name : String nummer : int
Student(String,int) + toString() : String

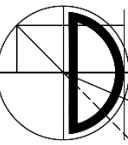
```
public class Student {  
    ...  
    public String toString() {  
        return "Student "  
            + this.name  
            + " hat die Matrikelnummer "  
            + this.nummer  
            + ".";  
    }  
} // Ende der Klasse
```



- Da wir im Methodenkopf den Rückgabetyt **String** vereinbart haben, muss unsere Methode natürlich auch einen Wert zurückgeben. Wir geben also durch die `return`-Anweisung das Ergebnis an die aufrufende Methode weiter.

Student
name : String nummer : int
Student(String,int) + toString() : String

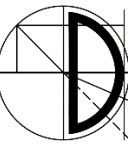
```
public class Student {  
    ...  
  
    public String toString() {  
        return "Student "  
            + this.name  
            + " hat die Matrikelnummer "  
            + this.nummer  
            + ".";  
    }  
} // Ende der Klasse
```



- Der Zugriff auf die Instanzvariablen des Objektes erfolgt hierbei wieder durch das bereits bekannte Schlüsselwort **this**

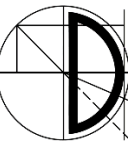
Student
name : String nummer : int
Student(String,int) + toString() : String

```
public class Student {  
    ...  
  
    public String toString() {  
        return "Student "  
            + this.name  
            + " hat die Matrikelnummer "  
            + this.nummer  
            + ".";  
    }  
} // Ende der Klasse
```



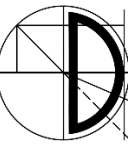
- Bei dem entstandenen Objekt handelt es sich um einen normalen String, den wir wie gewohnt über die Methode `println` auf dem Bildschirm ausgeben können

Student	<pre>Student charly = new Student("Kalle Karlsson", 978432) ;</pre>
name : String nummer : int	<pre>String beschreibung = charly.toString() ; System.out.println(beschreibung) ;</pre>
Student(String,int) + toString() : String	<pre>charly:Student name = „Kalle Karlsson“ nummer = 978432</pre>



- Die soeben vorgestellten Befehlszeilen lassen sich natürlich noch verkürzt darstellen. Wenn man (wie in unserem Beispiel) die textuelle Beschreibung nur in der Ausgabe verwenden will, braucht man sie natürlich nicht explizit mit einem Variablennamen zu benennen.

Student	<pre>Student charly = new Student("Kalle Karlsson", 978432) ;</pre>
name : String nummer : int	<pre>String beschreibung = charly.toString() ; System.out.println(beschreibung) ;</pre>
Student(String,int) + toString() : String	<pre>System.out.println(charly.toString()) ;</pre>



- In diesem speziellen Fall können wir sogar den Aufruf der `toString`-Methode wegfallen lassen.
- Grund hierfür ist die Definition der Methode `println` für Objekte. Diese ruft für das auszugebende Objekt automatisch die Methode `toString` auf und druckt das Ergebnis auf dem Bildschirm.

Student	<pre>Student charly = new Student("Kalle Karlsson",978432) ;</pre>
name : String nummer : int	<pre>String beschreibung = charly.toString() ; System.out.println(beschreibung) ;</pre>
Student(String,int) + toString() : String	<pre>System.out.println(charly.toString()) ; System.out.println(charly) ;</pre>

Schon gewusst, dass es auch „Anti-Konstrukturen“ gibt?

Während der Konstruktor eine entscheidende Rolle bei der Erzeugung von Objekten spielt, unterstützen manche Programmiersprachen (z.B. C++) auch die Definition von Destrukturen. Destrukturen werden ausgeführt bevor die Speicherbereich eines Objekts wieder freigegeben wird. In Java gibt es verwandtes Konzept namens Finalisierung. Finalisierer sind in ihrer Funktionsweise allerdings stark eingeschränkt, noch stärker als Destrukturen.



"It was much nicer before people started storing all their personal information in the cloud."

The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'J' and 'M' forming a square. To the right of the logo, the text 'Julius-Maximilians-' is in a smaller font, and 'UNIVERSITÄT WÜRZBURG' is in a larger, bold, white font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit